PYTHON ACTIVITY-BASED LEARNING

CASE STUDY 11: Examination Result and Grace-Mark Validation

B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Process marks across multiple subjects, apply subject-wise pass criteria and a specified grace-mark policy, and generate final classifications without exceeding maximum marks.

2. OBJECTIVES

- Calculate total and percentage from subject marks.
- Check subject-wise pass marks.
- Apply a limited grace-mark rule where permitted.
- Display final result and classification.

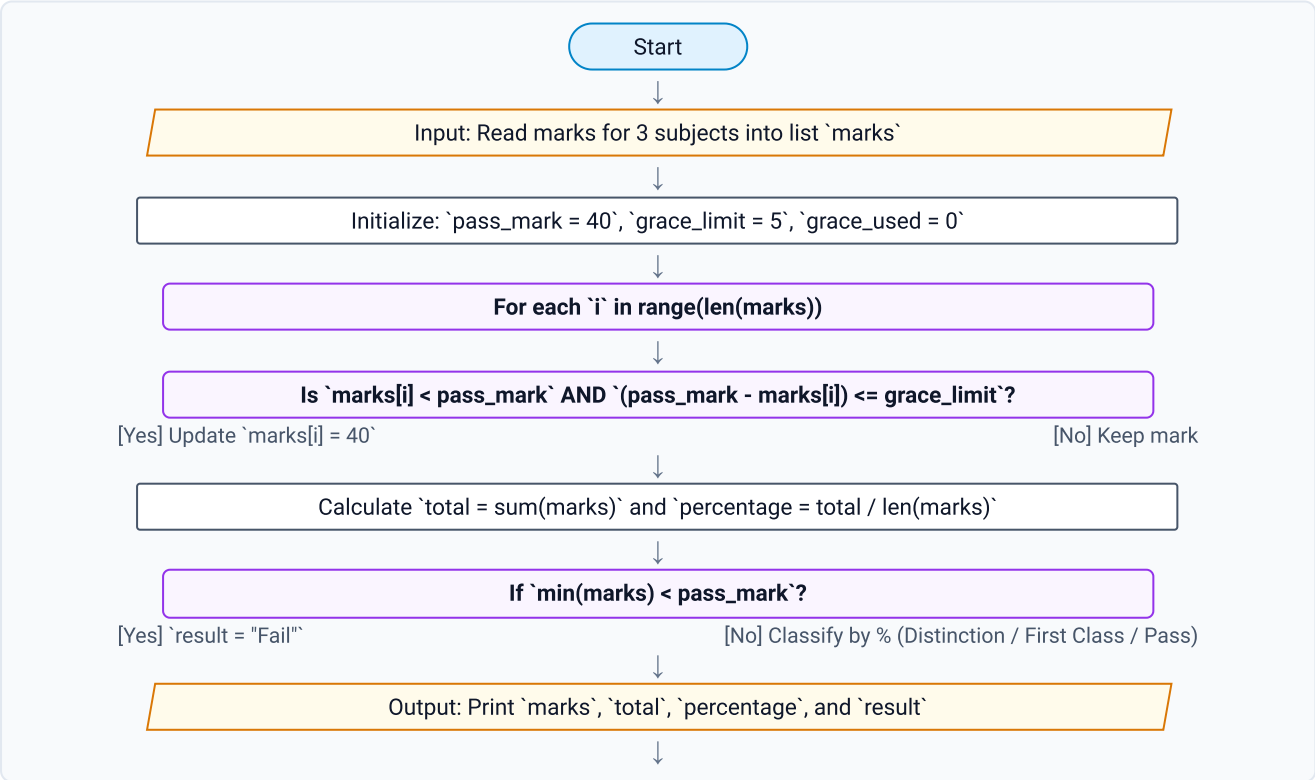
3. ASSUMPTIONS

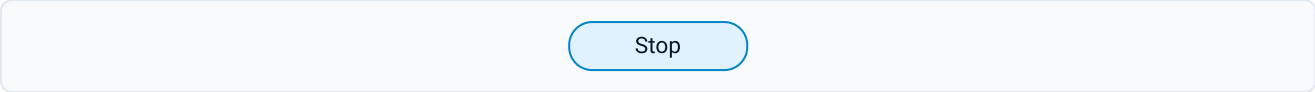
- There are 3 subjects, each out of 100. Pass mark is 40 in each subject.
- At most 5 grace marks may be added to one failed subject. Grace marks cannot make a subject exceed 100.
- Classification is Distinction at 75% or above, First Class at 60% or above, otherwise Pass.

4. PYTHON EXECUTION ALGORITHM

1. Initialize inputs `marks = [input1, input2, input3]`, `pass_mark = 40`, `grace_limit = 5`.
2. Iterate through array index `i` from 0 to 2.
3. If `marks[i] < 40` and `(40 - marks[i]) <= 5`, update `marks[i] = 40`.
4. Compute `total = sum(marks)` and `percentage = total / 3`.
5. Determine result: If `min(marks) < 40` set result to Fail, else classify by percentage.
6. Print `marks`, `total`, `percentage`, and `result`.

5. PYTHON ORDER FLOWCHART





6. PSEUDOCODE

```
START
  INPUT marks[0], marks[1], marks[2]
  pass_mark = 40, grace_limit = 5
  FOR i = 0 TO 2 DO
    IF marks[i] < pass_mark AND (pass_mark - marks[i]) <= grace_limit THEN
      marks[i] = pass_mark
    END IF
  END FOR
  total = SUM(marks)
  percentage = total / 3
  IF min(marks) < pass_mark THEN
    result = "Fail"
  ELSE IF percentage >= 75 THEN result = "Distinction"
  ELSE IF percentage >= 60 THEN result = "First Class"
  ELSE result = "Pass"
  END IF
  OUTPUT marks, total, percentage, result
STOP
```

7. PYTHON SOURCE CODE

```
marks = [int(input("Enter marks for subject 1: ")),
         int(input("Enter marks for subject 2: ")),
         int(input("Enter marks for subject 3: "))]
pass_mark = 40
grace_limit = 5

for i in range(len(marks)):
    if marks[i] < pass_mark and pass_mark - marks[i] <= grace_limit:
        marks[i] = pass_mark

total = sum(marks)
percentage = total / len(marks)

if min(marks) < pass_mark:
    result = "Fail"
elif percentage >= 75:
    result = "Distinction"
elif percentage >= 60:
    result = "First Class"
else:
    result = "Pass"

print("Final marks:", marks)
print("Total:", total)
print("Percentage:", percentage)
print("Result:", result)
```

8. TEST CASES & OUTPUT

Test	Input Scenario	Expected / Actual Output
1	Subject 1: 42, Subject 2: 38, Subject 3: 66	Final marks: [42, 40, 66] | Total: 148 | Percentage: 49.33 | Result: Pass

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING

CASE STUDY 17: Student Performance Trend Analysis

B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and classify students according to defined criteria.

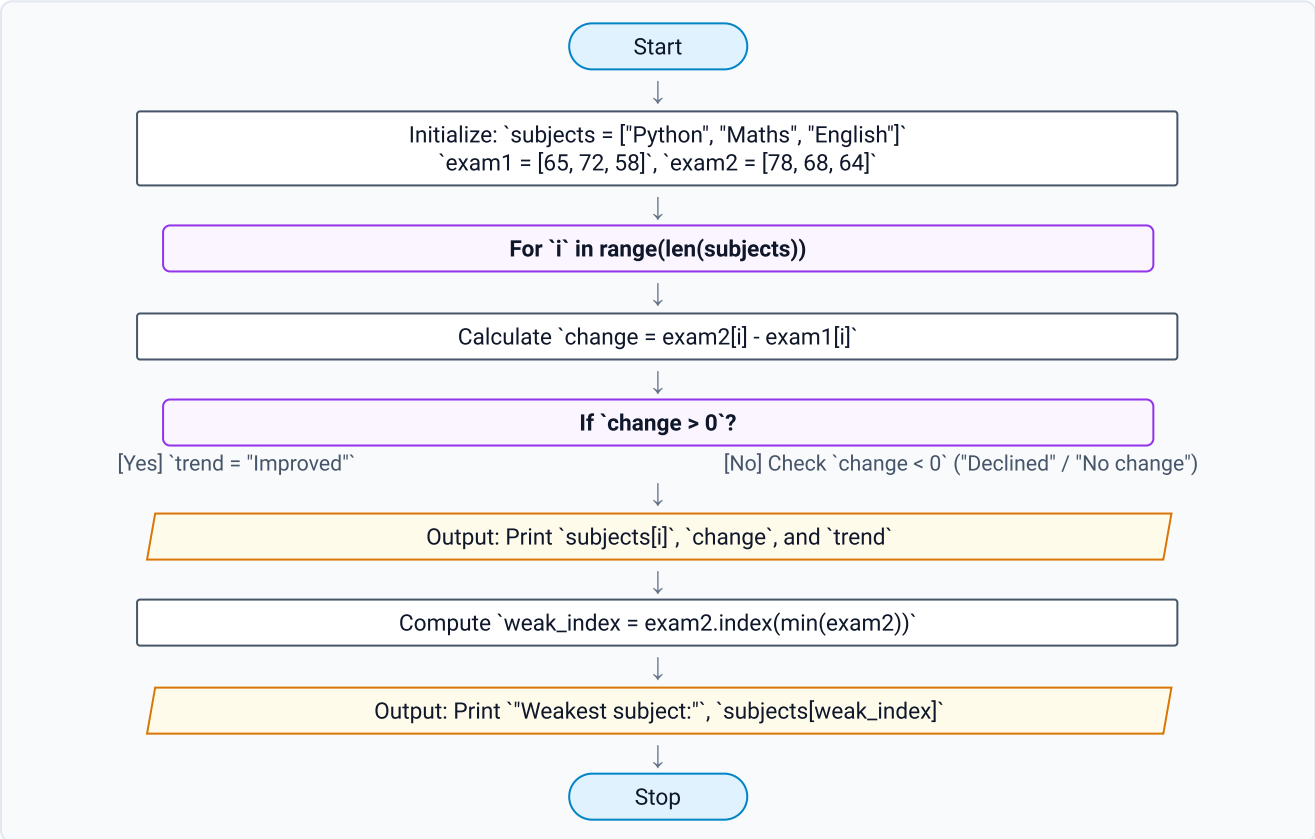
2. OBJECTIVES

- Store scores from two exams.
- Calculate subject-wise changes.
- Identify improvement or decline.
- Find the lowest-scoring subject.

3. ASSUMPTIONS

- Three subjects are considered: Python, Maths, and English.
- Each subject has marks out of 100.

4. PYTHON ORDER FLOWCHART



5. PYTHON SOURCE CODE

```
subjects = ["Python", "Maths", "English"]
exam1 = [65, 72, 58]
exam2 = [78, 68, 64]

for i in range(len(subjects)):
    change = exam2[i] - exam1[i]
    if change > 0:
        trend = "Improved"
    elif change < 0:
        trend = "Declined"
    else:
        trend = "No change"
    print(subjects[i], "change:", change, "-", trend)

weak_index = exam2.index(min(exam2))
print("Weakest subject:", subjects[weak_index])
```

6. SAMPLE OUTPUT

```
Python change: 13 - Improved
Maths change: -4 - Declined
English change: 6 - Improved
Weakest subject: Maths
```

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING

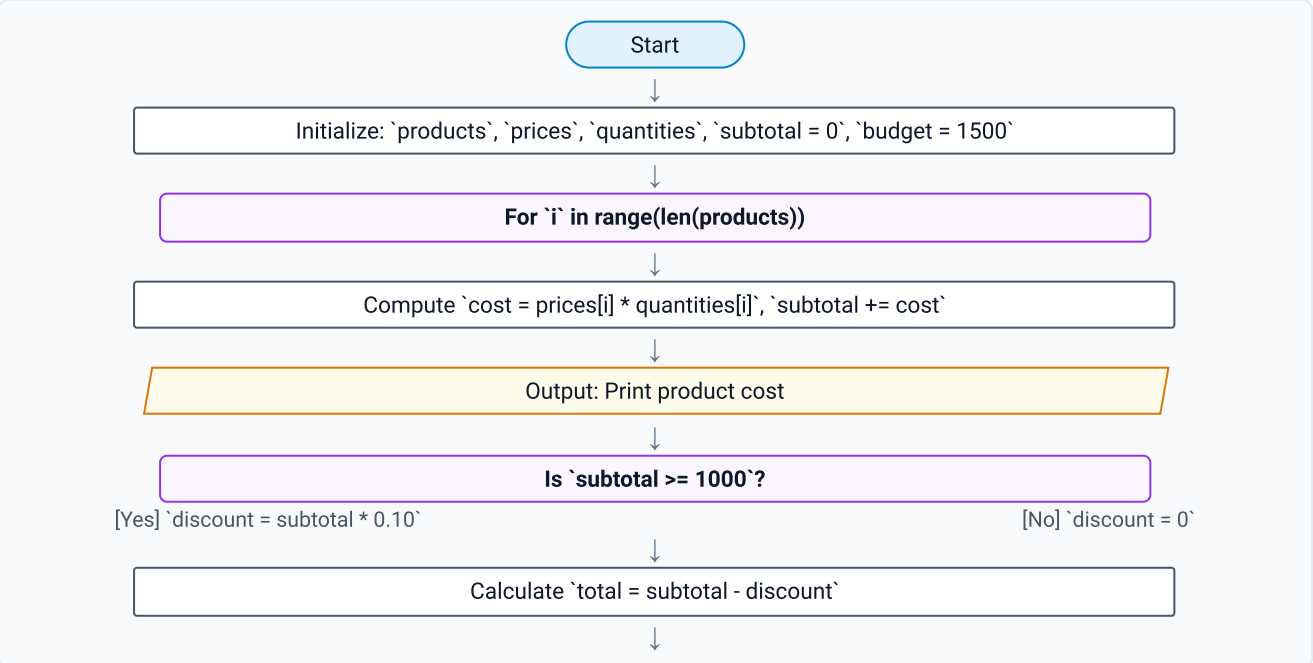
CASE STUDY 19: Multi-Product Shopping Budget Constraint

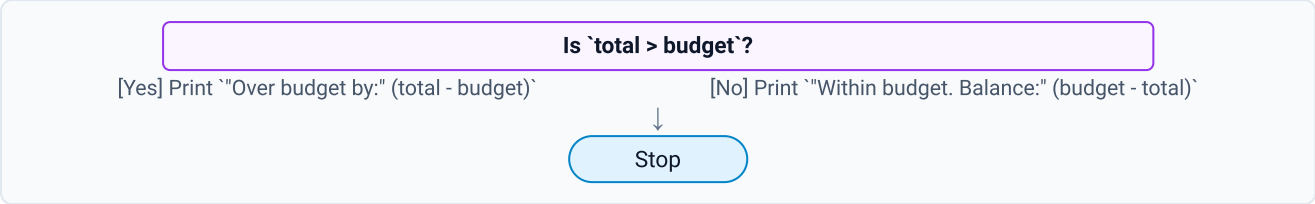
B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine whether total purchases remain within a fixed budget constraint.

2. PYTHON ORDER FLOWCHART





3. PYTHON SOURCE CODE

```
products = ["Notebook", "Pen", "Bag"]
prices = [80, 20, 900]
quantities = [3, 5, 1]

subtotal = 0
for i in range(len(products)):
    cost = prices[i] * quantities[i]
    subtotal += cost
    print(products[i], ":", cost)

discount = 0
if subtotal >= 1000:
    discount = subtotal * 0.10

total = subtotal - discount
budget = 1500

print("Subtotal:", subtotal)
print("Discount:", discount)
print("Final bill:", total)

if total > budget:
    print("Over budget by:", total - budget)
else:
    print("Within budget. Balance:", budget - total)
```

4. SAMPLE OUTPUT

```
Notebook : 240
Pen : 100
Bag : 900
Subtotal: 1240
Discount: 124.0
Final bill: 1116.0
Within budget. Balance: 384.0
```

Submitted by: _____ Roll No.: _____

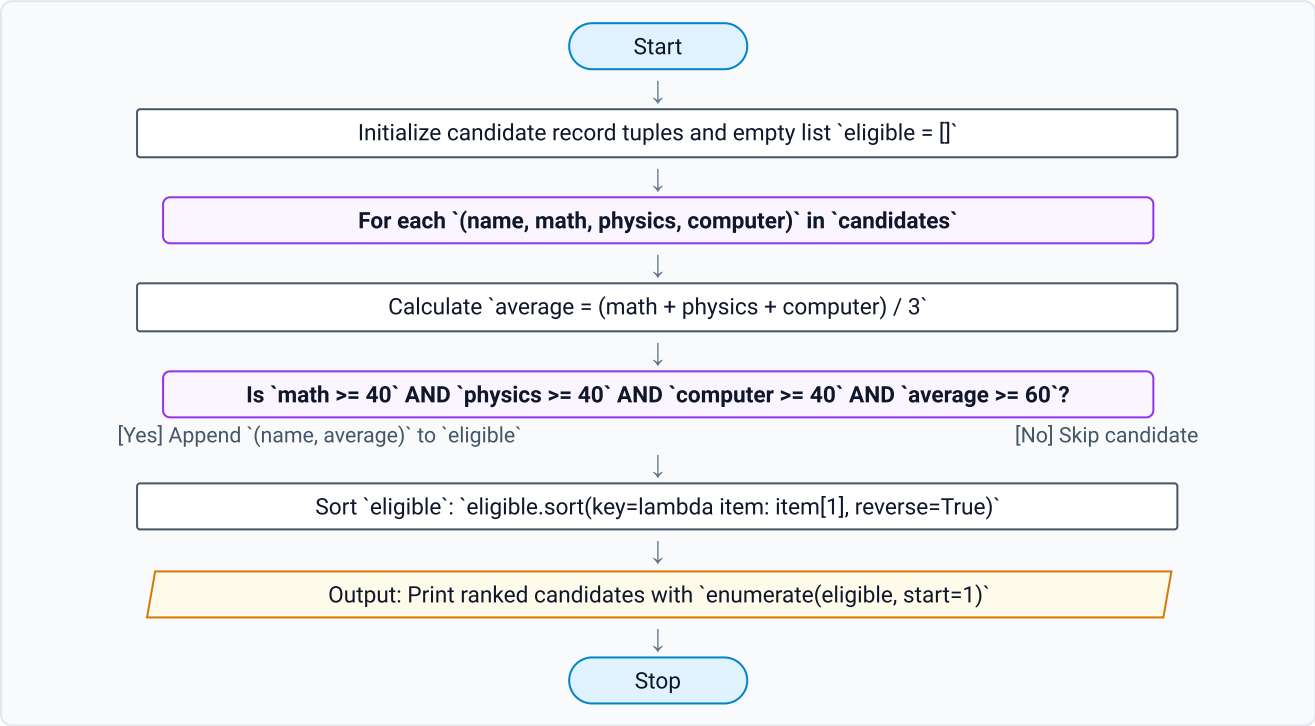
CASE STUDY 20: University Admission Decision Engine

B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Evaluate candidates using subject eligibility, aggregate marks, and ranking criteria; generate a ranked list of eligible applicants using dynamic list sorting.

2. PYTHON ORDER FLOWCHART



3. PYTHON SOURCE CODE

```
candidates = [
    ("Asha", 80, 75, 90),
    ("Ravi", 55, 62, 70),
    ("Mina", 35, 88, 91),
    ("Kiran", 70, 70, 70)
]

eligible = []
for name, math, physics, computer in candidates:
    average = (math + physics + computer) / 3
    if math >= 40 and physics >= 40 and computer >= 40 and average >= 60:
        eligible.append((name, average))

eligible.sort(key=lambda item: item[1], reverse=True)

print("Admission ranking")
for rank, candidate in enumerate(eligible, start=1):
    print(rank, candidate[0], round(candidate[1], 2))
```

4. SAMPLE OUTPUT

```
Admission ranking
1 Asha 81.67
2 Kiran 70.0
3 Ravi 62.33
```

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING

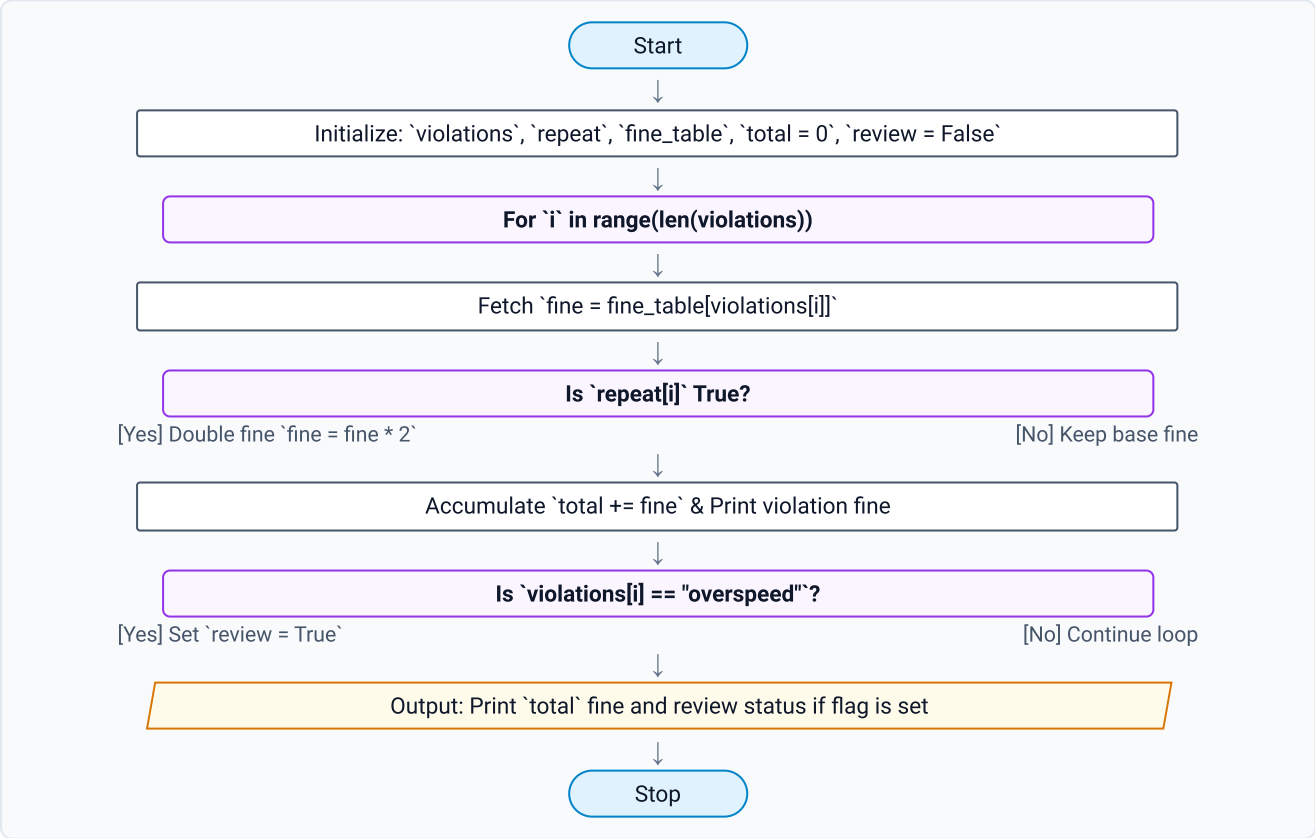
CASE STUDY 21: Traffic Violation Penalty Calculator

B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Process multiple traffic violations using vehicle category, offence type, repeat-offence status, and dictionary fine lookup tables.

2. PYTHON ORDER FLOWCHART



3. PYTHON SOURCE CODE

```
violations = ["helmet", "signal", "overspeed"]
repeat = [False, True, False]
fine_table = {"helmet": 500, "signal": 1000, "overspeed": 2000}

total = 0
review = False

for i in range(len(violations)):
    fine = fine_table[violations[i]]
    if repeat[i]:
        fine = fine * 2
    total += fine
    print(violations[i], "fine:", fine)
    if violations[i] == "overspeed":
        review = True

print("Total fine:", total)
if review:
    print("Status: Requires review")
```

4. SAMPLE OUTPUT

```
helmet fine: 500
signal fine: 2000
overspeed fine: 2000
Total fine: 4500
Status: Requires review
```

Submitted by: _____ Roll No.: _____

PYTHON ACTIVITY-BASED LEARNING

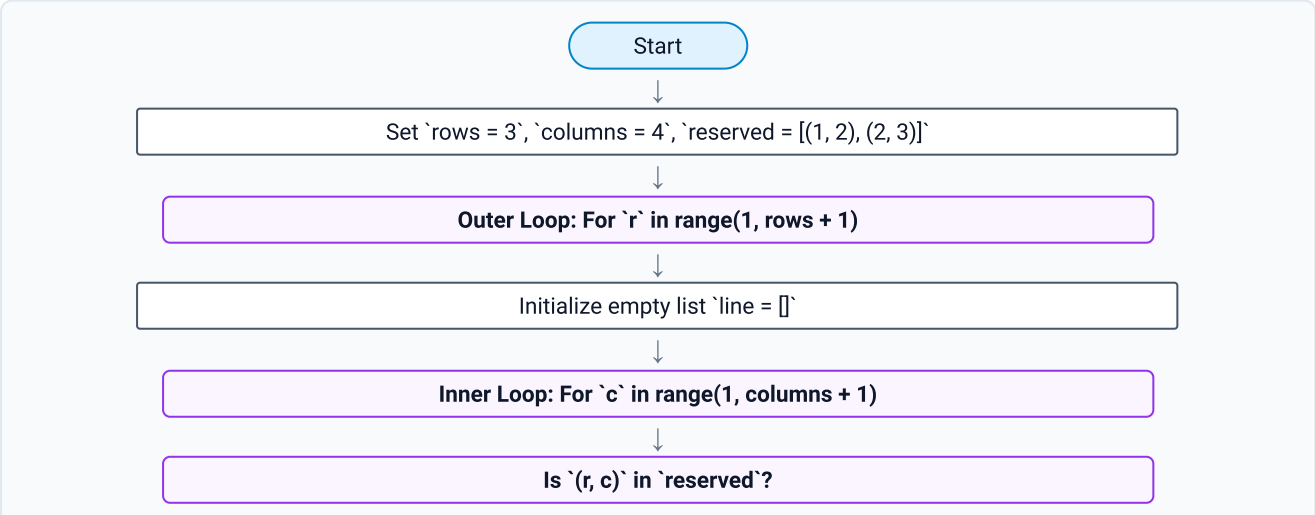
CASE STUDY 26: Nested Loop Examination Seating Generator

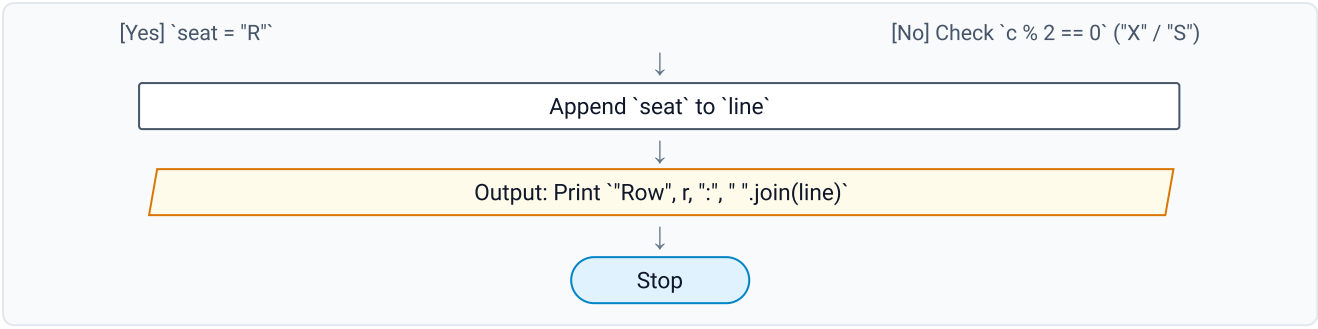
B.Tech First Year - CSE | Python Programming

1. PROBLEM STATEMENT

Generate a seating arrangement with row and column identifiers, reserved positions, and alternate seating rules using nested execution loops.

2. PYTHON ORDER FLOWCHART





3. PYTHON SOURCE CODE

```
rows = 3
columns = 4
reserved = [(1, 2), (2, 3)]

for r in range(1, rows + 1):
    line = []
    for c in range(1, columns + 1):
        if (r, c) in reserved:
            seat = "R"
        elif c % 2 == 0:
            seat = "X"
        else:
            seat = "S"
        line.append(seat)
    print("Row", r, ":", " ".join(line))
```

4. SAMPLE OUTPUT

```
Row 1 : S R S X
Row 2 : S X S R
Row 3 : S X S X
```

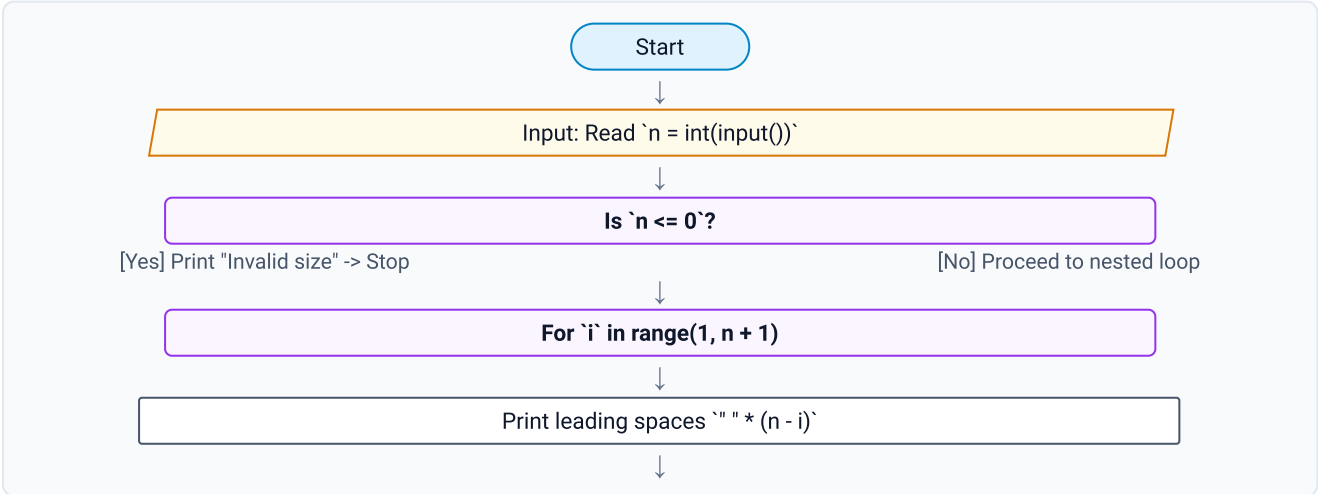
Submitted by: _____ Roll No.: _____

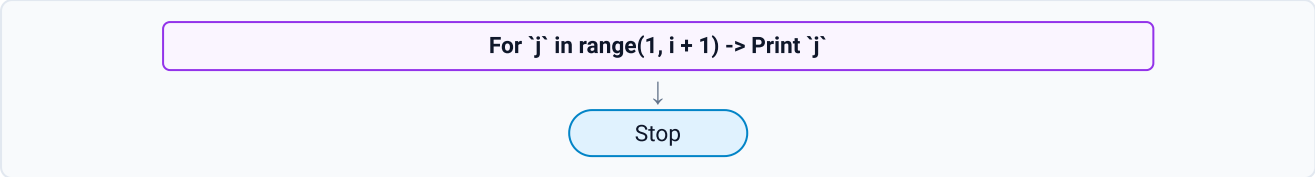
PYTHON ACTIVITY-BASED LEARNING

CASE STUDY 27 & 31: Numeric Patterns & String Evaluation

B.Tech First Year - CSE | Python Programming

CASE STUDY 27: NUMERIC PATTERN FLOWCHART





CASE STUDY 31: EXAMINATION ANSWER EVALUATION CODE

```
key = ["A", "C", "B", "D", "A"]
answers = ["a", "C", "D", "", "A"]
correct = 0
wrong = 0
blank = 0

for i in range(len(key)):
    answer = answers[i].upper()
    if answer == "":
        blank += 1
    elif answer == key[i]:
        correct += 1
    else:
        wrong += 1

score = correct * 2 - wrong * 0.5
print("Correct:", correct)
print("Wrong:", wrong)
print("Unanswered:", blank)
print("Final score:", score)
```

SAMPLE OUTPUT

```
Correct: 3
Wrong: 1
Unanswered: 1
Final score: 5.5
```

Submitted by: _____ Roll No.: _____

CASE STUDY 47 & 52: Ranking & Team Formation

B.Tech First Year - CSE | Python Programming

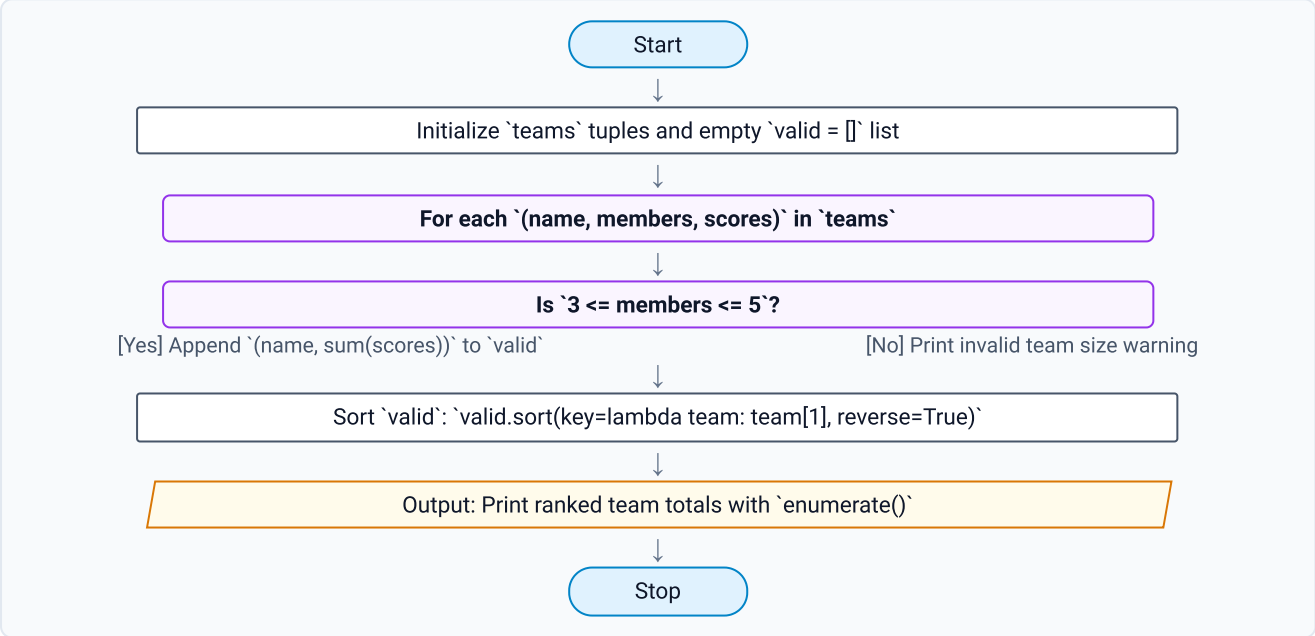
CASE STUDY 47: SPORTS TOURNAMENT RANKING CODE

```
teams = [
    ("Falcons", 7, 8),
    ("Tigers", 7, 12),
    ("Wolves", 5, 4),
    ("Eagles", 9, 6)
]

teams.sort(key=lambda team: (team[1], team[2]), reverse=True)

print("Tournament ranking")
for rank, (name, points, difference) in enumerate(teams, start=1):
    print(rank, name, "Points:", points, "Difference:", difference)
```

CASE STUDY 52: HACKATHON TEAM FORMATION FLOWCHART & CODE



```
teams = [
    ("CodeStars", 4, [80, 90]),
    ("ByteBuilders", 3, [85, 85]),
    ("LogicCrew", 2, [95, 90]),
    ("Debuggers", 5, [70, 88])
]

valid = []
for name, members, scores in teams:
    if 3 <= members <= 5:
        total = sum(scores)
        valid.append((name, total))
    else:
        print(name, "invalid team size:", members)

valid.sort(key=lambda team: team[1], reverse=True)

print("Valid team ranking")
for rank, (name, total) in enumerate(valid, start=1):
    print(rank, name, "Total:", total)
```

Submitted by: _____ Roll No.: _____